01 - lesson

Tags	
Course	🦠 Malwer Analysis
Confidence	Not Confident
Materials	MAIFMA-2026-02-Assembly.pdf
Last Edited	3 November 2026 14:28

All right.
So, as I was saying, welcome to this edition of the course.
So I'm assuming that the population is gonna be mixed, because this course is made available in two flavors. You might be here as a cybersecurity student, which means that you are enrolled in the Malware Analysis and Incident Forensics class. Could you raise your hand if you are from cybersecurity?
Aha.
And then you might be here as an engineering student.
Don't be shy.
Okay, a bit fewer than last year, but I don't get a full picture on the back. Okay, cool. If you're an engineering student.
We're here only for the malware analysis part, which is nevertheless the most characterizing part of this exam. And... so...
Indeed, we are made of 9 credits.
6 are on the malware analysis parts.
And 3 are for incident forensics.
And those 3 credits are... specifically for
yeah, cybersecurity student, but of course there are gonna be some lectures that might be of independent interest, and when it's the case, I'm gonna advertise them also to the engineering students. Maybe we'll have some seminar or some activity that might be, you know, of larger interest also to
So you... So, 9 credits. This runs from today till...
Three days before Christmas, allegedly, because this year the semester is gonna stretch until... December 22nd.
I'd be curious, but I'm being recorded, so I won't ask.
Yeah, where is this course? Well, we...
Okay, I should say, this year is... Fairly simple.
We are every day in a different room.

But, at least they don't change the room so much. We're gonna have only a couple of lectures that are gonna be in other nearby rooms due to institutional events. But primarily, every Friday we're here. That's the only thing that won't change, and has never changed since the very first edition of this course. So, Fridays.
all students. So if you are cyber security or engineering, or you can be also from another program in principle. And you choose this as an optional exam. Well.
In any case, we are here every Friday from 2 to 5.
Then...
The same applies to Tuesdays. The Tuesday's lectures are for both, 9 and 6 credits, right? And it's a 2-hour lecture. We start at 2.
And we are gonna be in Regina Elena building.
Building the Room 101.
And on Thursday, that's only for the 9 credits, so on Thursday, we do the incident forensics parts, and occasionally some seminars.
Which are of broader interest, so they are on topics that they span bots, let's say.
Major topics of this exam.
So, in that case, we move one floor up, we go to room 201. Alright, so I think there's gonna be just so far to announce only two changes.
About a long, like, 12, 13 weeks, so I guess we're lucky. Last year, we were split between
Via Saladian here.
Oh, it was quite a nightmare for logistics, especially for you.
Which is my primary concern, but at least now we are all...
We are close. So, who's the teaching team?
Since its first edition, it's been my colleague, Leonardo Quercioni, and I.
So, for some time...
one person is doing 6 credits, and then it's another person, so in the last couple of years, it's been me. I will be doing 6 of the 9 credits, but it doesn't mean that I'm gonna do malware analysis and my colleague is gonna do the other part.
No, actually, we are co-teaching this, so we're all... both of us are gonna give lectures in
across the 9 credits. So, actually, today you're with me, next Friday you're with me, but next week you're gonna be with my colleague.
And... yeah.
So, if you... probably you've met either of us before. If you haven't, well, okay,

I'm a tenure track assistant professor at the DIAC department, the engineering department.
I'm a security researcher. I specialize in program analysis to
Optimize and solve security problems, such as...
Finding vulnerabilities at scale, analyzing software that doesn't want to be analyzed. One of the reasons why I'm teaching malware, malware analysis courses.
And, together with Leonardo and another colleague, we co-founded a software and system security laboratory here in Sapienza. We do...
yeah, I'd like to say that we do cutting-edge research on these topics. Some of this research will be shared with you at some point, probably we will pitch some of our own
projects and stuff.
We... occasionally we've done some research, yeah, quite often, I would say. And even successfully.
frequently, often, with master's students, so that they led to breakthroughs in the field, publications. We had before even not just academic stuff, but we had students that later presented in
Blaketti events, or RICON, if you're familiar with those conferences. So, I mean, we are academics, but we are very much involved.
In the cybersecurity community.
But this is not about us, but it was fair to share who we are, so let's move to the rest of the course. Well, the general information, I think that's... I have, yeah, a couple more slides on general information.
We do use a textbook for this course, but you don't need, necessarily, the textbook. It's a good reference, it teaches you some methods.
the book is old, which means that even some of the tooling is old, but these principles remain the same. So the book, I think, is, like, 15 years old, then they made an update and stuff, but the principles, the way that people do malware analysis in the field hasn't changed. We got better tools now.
But, the way that you dissect malicious software, like the title of this book, remain pretty much the same. It's, fun.
and challenging process.
Where you're essentially... you're... you're faced with creativity of other people. We're gonna see that malware
Malicious software is not like the ordinary software that you have developed or even tried to analyze before. It's something... it's one of a kind, and so there are specific ways of approaching this, and there is a lot of creativity on the side of

people who write malware for a living or for fun.
I mentioned classroom. Classroom is important because you're gonna find the slides, these recordings, and I believe that the slides, the recordings, and the exercises that are gonna share
we're gonna share on the platform, they are more than enough to successfully pass this exam with a very high grade. So, the materials that we provide
Are more than enough.
But we will provide occasionally some additional materials, like papers, if you want to dig on a specific topic.
And, you know, pointers or announcements to...
activities as well. Classroom is gonna be a bit our... contact platform.
And you're gonna find some tooling there.
Specifically, we prepared a virtual machine. It's the one that I'm using right now for, you know, hosting this lecture. It's called MAIF, like the course, M-I-M-A-I-F.
And it's a Windows machine that has been prepared to ease malware analysis.
This means that... It's a machine, well, first of all, that you shouldn't be using for purposes other than this course, because it's...
deliberately outdated, so you don't have some Windows protections that get in the way when you want to analyze malware, but if you don't want to analyze malware and you want to use it for your daily stuff, then, I mean, you're just, you know.
Preparing, rolling our...
carpet for people to go and, you know, attack your machine, because it's gonna be vulnerable, it's outdated, but it serves the purpose well. So it's a Windows 10, I think it's a build from 6 years ago.
It should be the initial build. We have the updates disabled, there's no defender active, it's, you know, it's a specifically configured machine.
And to run it, You're gonna need... a virtual machine manager. Now.
The tool of choice is VirtualBox.
It's the tool that we use for developing and testing this machine.
If you're a Mac user.
Now, you see this... I mean, you see me with this computer, but I do own a Mac, and it's an ARM.
You might have a bit of a problem, because you're gonna need an emulator, and it's gonna be slow. You can make it work.
Especially if your MacBook is fairly recent, so an M3 or higher.
It's workable. It's not ideal, but it's workable.

In principle, I mean, if you come here every Friday, you attend the labs.
And you do some extra on the side, you're gonna be good.
But just saying that.
there is a dedicated procedure in case you have a CPU that is not an Intel or AMD CPU. So you're gonna need a different set of tooling. Everything will be linked on
classroom for the next Friday. I mean, honestly, I think I'm gonna share it later today or over the weekend, but just so you know, you're gonna need this virtual machine. Now, in the past, I mean, 5 years ago, I was getting much fewer complaints than I've been getting in recent years, because people have much faster
Intel or AMD notebooks, but somehow they mess, or, I mean, both them and the default hypervisors configuration, they mess with performance. So, let's say that these virtual machines were running
smoother in the past than they are now, on less powerful hardware, and that's one of the fun things of technology. But, believe me, these VMs are meant to be fast on your machine, so whenever you have a problem.
I suggest that.
Well, you make sure that you have virtualization enabled, go through the normal troubleshooting guides that you will find online. Maybe you have the virtualization from Windows 10 or 11 that it's interfering with VirtualBox.
So this might be a problem, but there's plenty of guides online.
If you're a Linux user, this is gonna be easier for you, because Linux hosts VM just fine. If you want, you might even run this in KVM, if you're not big on VirtualBox. So you can import this and it should run just fine. Yeah, then we're gonna...
Yeah, I think we're gonna have a laboratory session, even on bringing up this virtual machine, but anyway, it's important that you get familiar with this. Maybe you create a snapshot, so you have a clean state of this machine.
Because we're gonna be messing with this machine a lot. When we experiment with malware, sometimes we corrupt the state of the machine, we want to go back to the normal state. It's part of the process, it's not a normal software that we're running. So, during the exams, for example,
What happens is that, you know.
To show that you achieved some progress in the exam at some point.
the exam software that we give you to analyze is gonna change your wallpaper or show funny messages on your system, you may want eventually to restore your snapshot. I'll get to that part, which I believe is the very next slide on how

we are gonna evaluate you.
So...
Right, so if you're here for 6 credits, you have to read only the first part. If you're here for 9 credits, then the exam is in two parts. The part that is most important for the grade, for the final grade, is the first, the one that is shared between the two exams.
We have a practical test.
You might expect it to last.
Up to 3 hours, not necessarily 3 hours, but imagine it is a pretty long exam, where we're gonna give you a malware sample, something that we created, it's a synthetic malware sample. Every time it's different, we change the techniques that we implement and the behavior of the sample, and you're supposed to analyze this sample.
and comment on specific questions that we're gonna ask you about this sample. Okay.
And on this, you're gonna get a grade that is a number between 18 to 30, if you pass.
Otherwise, you know, you have to do it again. Then, there's another part that's only for the extra 3 credits.
This part doesn't change the final mark.
So it's more like you have to pass as a Boolean.
Check this exam.
this other part.
If you do it, And you pass also the practical part, you get your grade.
We have some flexibility on this, which means that
If you show up, and you do fairly well at the practical test, but for some reason you do poorly the questionnaire and you don't pass. You don't have to redo the practical test. And also the other way around.
If you pass the questionnaire.
But you do not so great at the practical tests, we're gonna keep a positive outcome.
There's one bot. This is contingent on doing a fairly reasonable exam on the other part. If you come here and you hand in a report.
with no answers, or very little content, I'm assuming that you're here to cheat on the rules, and you studied only for one part. If you do that, we're not gonna keep the outcome valid.
But if you come, and I don't know, you fall short because you get 12 out of 18.
for the minimum. That's okay. Doesn't mean that you were here to

to cheat. You just had a bad day, or the... you weren't prepared as you thought you were, I mean...
Let's say if you play it fairly.
You get to keep.
the other half of the exam, because we don't want to, you know, reassess your qualifications and your preparation. If you pass it once, it's enough, as long as you don't try to play the system.
Let's see, moving forward, the final... oh, they're endless, these general information slides. Obviously.
Yeah, it is the final one, yes. What will you learn from this course? Okay.
Malware. Malware is... You know, makes us.
In one word, the expression malicious software. It's software that is designed to harm.
And... This is a primary weapon in cyber attacks.
So, I mean, malware as... actually, the existence of malware predates cyber attacks. In the past, they were malware created just for fun.
for spreading to... within a network and just messing with a computer of your colleague or friend just for fun. There was no idea of attacking an organization, but nowadays, when you attack an organization, an entity, or a private individual.
That's called a cyber attack in the computer science domain.
And we want you to help understand the basic characteristics of cyber attacks that are done through malicious software.
You will learn to... Recognize and understand the basic structure.
of a malicious malware sample.
We're gonna teach you to analyze malware.
It's a... it's a process. So, initially, you're gonna acquire
Some basic knowledge using some fairly simple tools.
And these tools, they help you conduct two kinds of analysis. They're called static and dynamic.
Static means that you inspect this object.
without executing it. So I just take a good look at it, using a bunch of tools.
Then there is dynamic.
Which means you try to execute and observe how this sample behaves.
And one classic example is when you use the sandbox as services, which probably
several of you have heard before of sandboxes, so you... you take an untrusted software, and you ask this sandbox service to detonate this software for you,

they run it in a controlled environment, and they track what are the actions of this sample. Well, we're gonna learn that it's not so easy, because
First of all, malware.
Can be designed with... This is the... with these tools in mind, meaning that my fight back.
They might not want to be analyzed, they might try to deceive a sandbox, and behave, maybe.
Really nice when running a sandbox, but when you do it on your laptop.
That's gonna, you know, wipe your data, steal them, or worse. So, we're gonna learn...
that there are some anti-analysis techniques, and what to do about it. Well, what to do about it means moving
from this basic level of analysis to more advanced analysis. So we're gonna do advanced static analysis and advanced dynamic analysis. Advanced static analysis will mean understanding the code. So taking a look at the code in detail.
of the malware, and try to understand the behavior. Still without running the thing.
And then we can get to run it, and this qualifies for advanced dynamic analysis.
We don't run it in a sandbox, we were gonna run it in a tool.
Where we, the human analyst.
are able to trace step by step, even instruction by instruction, what the sample is doing. So in the second half of the lecture today, we're gonna get familiar with the language that you're gonna need.
to do this advanced static and dynamic analysis. We're gonna do a crash course on assembly language in the x86 domain, and this is gonna be...
covered over the next... another couple of lectures on this topic, just to brush up your x86 skills, if you already have them, or if you're new to this, you're gonna acquire them.
Just so you know, it's not a programming crash course.
I want to teach you how to read code, not to write. Now, this seems, I mean, seems okay, yeah, whatever. No, there's a fairly big difference between, you know, reading a language and writing correctly in a language. We experience this every time, even with normal, you know.
Normal languages for human communication, let alone with programming.
And, so I'm gonna teach you how to interpret what a piece of code in x86 assembly is trying to do.
But writing that code is a different exercise. It's more lengthy, a bit more

complex, more error-prone, but you don't need to do that.
We mentioned already the anti-analysis techniques. Now, with these topics, so far, we're gonna cover the six credits. Okay, those that are in common between the two.
The two courses.
When we give you this overview, we're gonna cover a lot of techniques that malware authors use. So, when I tell you that we're gonna do dynamic analysis and static analysis, we also need to know what we are observing.
So I'm gonna show you, with my colleague, some that are the typical traits and behaviors of malware. For example, malware might try to steal your data. We're gonna recognize that. Malware might try to acquire a foothold in your system, so to ensure its survival.
And it's called persistence, so we're gonna study the persistence technique. Or it's gonna infect other processes, that's called injection. So we're gonna cover all these techniques from the analysis perspective.
Okay, then if you're up for it, you can even develop some. Sometimes we're gonna discuss even implementations, but that's the 6 credits, right? Then, for the additional 3 credits.
It's the incident forensics part. You can expect two main learning goals here. The first is that you might understand the basic concepts of threat intelligence. That's a very important discipline.
My colleague is gonna give most of the lectures on this topic, starting already as of next Thursday, so you're gonna...
get familiar with threat intelligence, I will leave to him the introduction of this part. And the second learning goal is that we will learn how to recognize and extract something called indicators of compromise, IOCs.
from security incidents.
And those are indicators that might help you characterize a threat. For example, there are some malware samples that connect to specific domains or families of IP addresses, or they leave some particular traces in their victims that help you recognize them
And, you know, when you recognize that it's a known threat, you can look up reports online, you can find knowledge.
On how to best.
try to approach these threats. Right. Okay, final bit, all this will be about Windows malware.
Windows malware is predominant.
In terms of extent.

Most malware targets desktop machines. There's plenty on mobile.
But it's not that the techniques are so different. So the methods that we're gonna learn and present.
They are pretty broad. They are general in that sense. Now.
we only have 60 hours for malware, so I cannot realistically cover Linux for IoT malware, or Android, or iOS. Each of them has
some, you know, there's a curriculum, some knowledge that is about how these specific technologies run and are implemented, but the methods are the same.
So, whenever you want to do dynamic analysis, the advanced one, you're gonna need a debugger.
And there is a debugger for IoT stuff, there is a debugger for, cell phone, or whenever you're designing a sandbox, the principle is the same. Now, how you implement that is a different story.
But again, this is university. We are not in a company where the focus is, you know, to teach you how to use one specific tool.
We want to teach you, I believe, I mean, in general, in the university, we try to teach you a method, and we try to be grounded to reality with a specific class of application systems where we showcase this method in action. Bottom line, we're gonna do Windows for the entire course.
Because of its relevance, because of its history, and also, let me add, for the sophistication. Windows malware is, surely more sophisticated than any other malware you will see, for sure, on IoT devices, and the same can be said for most, but not all, mobile malware.
So, whenever you're dealing with something that is... you're gonna see obfuscated, well, beware that Windows obfuscation is typically much stronger, and so on.
Just because it's been running on for... from the 90s, pretty much. And the other... the other platforms have a fairly, fairly younger history.
Right. There's also...
Yeah, there's a disclaimer to share with you, because the techniques that we're gonna discuss, and some things that you're gonna learn.
It's stuff that can be, and most typically is used, to build malicious software meant to harm entities, which can be people, governments, companies. So, we're gonna...
approach them from an analysis side, but it's... yeah, it's reasonable to say that at the end of this course, if you do some research on the side, you might be able to develop stuff of this kind.
And this is where you have to make responsible use of your knowledge. I can

only say that
in... when we do computer security research, we have a lot of ethical guidelines. So, I've written malware before, I've reported and found issues, and Yeah, even in... very popular software, including Windows. My approach was not, you know, go on a forum and brag.
was contact the vendor and say, look, I found these problems.
How do we approach them? Do you have a plan for fixing them? How can I help? That's the responsible way.
of doing this kind of things. Then, you know, there's incentives on doing this for... Oh.
for profit, for any other reason, but you're entering a very dangerous area there, with lawful implications. So, yeah, bear this slide in mind. I said my piece. I think it was important, especially the ethical part. So when you do... when you approach these topics from research.
Okay, this is malware, but for example, I also do a lot of research on software testing.
And often in my projects with my students, we found a lot of vulnerabilities. And they spend a lot of time in reporting these vulnerabilities to the authors of this software. But, you know, you can even just say, okay, I've done my part for my project, I found these bugs, I'm happy. Well, no, there's still more work to do. You have to tell people that you found that problem.
And that's part of ethical guidelines. Now, the second thing that I want to discuss is that whenever you're analyzing, instead, malicious software, now, let's leave for a moment the hat of MalwareWriter. Let's go back to Malware Analyzer. When you do that.
It's software that has been written by people that you don't know, not your instructors in general, so this software can harm you in several ways. Normally, in a digital sense.
So, if you don't do this right, you may get that your machine that you use might be wiped.
There's a whole set of precautions that you must take when you try to analyze malware. Now, the samples that we will give you
they are designed to be harmless. They might prank you, that, yes, you're gonna see funny messages, or not so funny, but there's gonna be some prank part, but it's never gonna harm you in the sense of stealing your credentials or stuff. But when you analyze real-world malware, beware that you're gonna need a whole set of
sanitization measures. So you're gonna need dedicated machines that you

should clean every time, restore to a safe state. Otherwise, you might compromise your assets, okay?
Now, just a brief overview of some historical parts.
I want to show you just a perspective on how things changed.
So, a couple years before we started teaching this course, I think the first edition was 2018 or 19, anyway, a couple years before.
The main focus, or the thing that would hit the headlines, was data breaches.
So I started maybe the most, yeah, one of the largest was Yahoo. They lost data on half a billion people. This was huge. And Yahoo say that they were state-sponsored hackers. We're gonna get to that part in the future, but just so you know, there's many malicious actors that are working.
Sponsored by states.
as part of
Yeah, dedicated groups with a very high level of sophistication. So it started 2016, okay, big data breach, data stolen.
And then... Yeah.
There were some... Responses, yeah, we are aware of this, we're trying to protect this, that.
But there was internally in companies... now, here the, I mean.
Here, the slides are about Yahoo, but this has happened a million times over in many companies, and most of these times, we don't even know about it. But in the past, and even now, there's a struggle between the security measures that you're willing to implement, and this comes with a cost.
And the business decisions, where you want to save on costs.
So, there is a very thin line on how much you can
afford not to implement security measures, and sometimes, when you don't do, things can go bad in a very
big way, very wrong, and this might learn... I mean, this might drive your company to failure, to bankruptcy, because, I mean, you lose the most important thing you have, customers, because you break their faith in you. So, at that time, you know, it was still the...
The mood was, yeah, okay, we were working on it, but internally.
People were saying we made them aware of the issue a long time before.
And for U.S. citizens, this was very relevant just a year after there was this Equifax data breach, which exposed, sensitive data regarding financials, so, like, 160 million people, pretty much half of the... yeah, more than half of the adult population.
In the United States.

And it was, something that was a very known vulnerability that has been left unpatched, and
Some of these issues were already known two years prior. Okay, this is still the years where the concern is data breaches. Let's see how things changed.
In the years to come.
If I only find the right scrolling direction, yes.
So then, there was this very interesting event in summer 2016.
Where on the dark... on the dark web.
A group of hackers called shadow brokers.
Set up an auction, saying, hey, we have a software package of tools, for... harming systems.
So, we are selling very advanced software for system penetration.
And they say that this software was stolen or was coming from a group that is allegedly tied to the NSA.
So... They set an auction that was kind of weird.
There was no buyer that came forward. For what I recall, the price was insane.
I think... My feeling at the time is that they were trying to prove a point.
that this thing existed.
He goes after a while.
They release it to the public, all of it.
So these tools, they became available.
and There were a lot of zero days.
Being exploited, so there were bugs.
That's... were discovered.
Not reported to the vendors.
And instead of reporting, what I did is that I weaponized those bugs. So, meaning that if you had a vulnerable software, you knew that you were running, I don't know, this particular Windows version.
They could target you, they could attack you, so they didn't contact the vendor to say, you have a problem. They said, there is a... to themselves, there is a problem, let's use this problem to spy on these people.
And in this package, when people, you know, analyze it.
the kind of sophistication made the... I mean, was supporting the claim that there was some
Yeah, let's say that the source, NSA-related, was very credible. So the kind of tooling that you will find there was really, really advanced.
And some of this tooling.
Turns out that was... used in the very infamous WannaCry ransomware. Now, I

don't know if you remember this.
But there was a spring where.
I still recall on that day, I was, it was a Saturday, and we were doing this cyber challenge program in Sapienza. It was one of the very first editions, and this thing hit the headlines, where many British UK hospitals, they were halting to a ground because their systems had been compromised.
And what they did is that they brought a virus, computer virus, malware. that was spreading in networks using one of these vulnerabilities that were present and exploited by the package shared by shadow brokers.
So, it was one of the alleged NSA material. They used just an exploit, it was called Eternal Blue.
was a vulnerability in the SMB file system.
And this thing spread really fast.
Just because defenses, they were not anticipating a thing that will spread so fast, so indiscriminately, among whatever machine was in the network, you would attack it. And do what? Well, encrypt the files.
And ask for a ransom.
So, if you want your file back.
At that time, it was cheap. How much were they asking? I don't recall, but it was little.
like, $100 or something, or a thousand, was not much. Compared to what we're gonna see now for ransomware nowadays, it's... everything changed. But at the same time, this malware was not so sophisticated, so all it took to stop it.
Was registering a domain.
a website.
Because inside the code of WannaCry, there was a check.
So if a domain... Existed, this thing will not spread.
So it will stop replicating. And there was one guy who said, hey, I registered the domain, I stopped the infection.
For how I recall later, this guy ended up in a whole lot of other trouble. Not for this, but he had done illegal stuff before. I stopped following the story, but I do remember him going to jail.
And, you know.
I think we have a very valuable lesson here, that whenever you take software, like those NSA exploits, and you make them available to the masses.
you can come up easily with big threats. Now, in that case, people somehow got lucky, and thankfully, these hospitals and staff eventually were able to recover, just because the sophistication of what they brought was not so high.

Now, imagine this reaches capable hands, and unfortunately, most of the time it does.
you have a problem. And even worse, now we have tools, generative AI, which has a throughput much higher than any of us can afford in this room. So...
You can get extremely advanced technologies that become easily reachable.
The ransomware, just to conclude the story on ransomware, now, at that time, they were asking you, give me money for... have your data back.
And this money was in variable amount. Initially, they were asking in dollars, then they moved to Bitcoins, then do other stuff, then maybe you will pay and you will not get your data back.
And then this became a successful commercial model. They say, but you can do...
different levels of extortion. I mean, one thing you can say, look, I have your data, and you have to pay if you want it back. But I can say, I have my backups, I'm one of these very thorough people, I do a backup every day. Well, imagine now you're a law firm.
or you have sensitive data on people, I can say, you know what, I'm gonna take your sensitive data, and I'm gonna publish it online. I'm gonna leak the books.
Of all your customers.
All their budgets, all their stuff, unless you pay me.
So you can do another kind of extortion. You can say.
You have to pay me to not share your data, to stay silent on your data.
And in other cases, they might say, you know what, we might contact each of your customers and pressure them, and say, hey, we have your data that we acquired through them, so you do a double damage, and you can extort all the third parties. So this is, like, really...
Profitable, and people who are in crime are very smart, unfortunately.
So they saw a very good business opportunity, and some ransomware, they started as a toy, like WannaCry, almost a toy, and they became, like, a sophisticated ecosystem.
where you will have... here I'm discussing LockBit as an example. So there was, at some point, even a customer care.
that you can say, look, I want to have a variant of lock bit that has to hit these people.
And can you make it customized for me?
And essentially, it becomes ransomware as a service.
Just like you have infrastructure as a service, or you have storage with from cloud providers, so you rent CPUs from Amazon and others, you could rent

And obtain a run somewhere.
And they will have customer care, but even more nicely, they had a bounty program for bugs. So if you will find a bug in a ransomware, and you will say, hey, you have this problem, you can fix it in this way, you will get money from these people. It's quite, I mean, it's mind-boggling, I find it.
Final bit on this.
or horizon description market.
How is this relevant? Now, one thing is the crime.
But how about for defenders? I'm not a big fan of those slides, but I just want to show you the progression, so I picked just one. This was the malware analysis market in this projection from a couple years ago. Look at how
the sides changed. So this, like, quadrupled, something like that, yes, it did, from 2017, now it's at least... In 2024, predicted was
Like, 4 times more, if not 5 times more, than it was in 2017.
A look at the proportion. Initially, it was primarily North America and Europe, these bottom two bars.
But now, there's also the other half of the world, which has become very relevant.
So, this market is expanding, and there is need
for specialists in all sorts of places. So, whenever you want to work in malware analysis at a different level, there is a huge market out there. So, the things that we're gonna see in this course
They have practical Relevance, also for landing a very successful position in industry.
And it's nice to hear that pretty much for every, I would say, my experience, at least a couple of people every year that graduate.
from this class, in one or two years from getting their degree, they ended up doing very nice jobs in malware firms. So... yeah, I mean, maybe I will share some stories in future of
projects or research that has been done by students, not with me, but just in their professional life following graduation, and it's very rewarding. From my perspective, it's a lot, but I will say, as a peer, people who studied with them, it's a really nice thing. Now.
Let me conclude this first half of the lecture with something that is now more about the terminology.
So this terminology, we're gonna need it now. We're gonna, I would say, rip this band-aid off. We're gonna learn this infamous... it's called the kill chain. Have you seen this before?

And some hands, yes, okay, some people have already, and unfortunately, I will say, met this thing. No, okay, it's not that bad. Honestly, it has a lot of value, this model, for sharing
with non-technical people, some concerns for communicating within agencies, within organizations. So, this was something borrowed from the military jargon. It's called Kill Chain, or Intrusion... Kill chain.
And was modeled by Lockheed Martin. If you don't know Lockheed Martin, among others, they developed military aircrafts. They are very huge.
Yeah, very huge company.
In the order of billions and billions. Of...
The stuff that they build. So, they modeled this process, this intrusion kill chain, wants to model how, in general, perpetrators, they carry out a cyber attack.
Okay, and identify a number of stages, which start with Recon, so reconnaissance stage, where you're gonna explore
The boundaries of your victims probe their vulnerabilities, their weak points. Maybe they have some vulnerable software, or maybe the vulnerability is some person that we're gonna target through social engineering, phishing emails.
Any sort of scam.
You identify a weak point. Then you need to weaponize this weak point. If it's a software.
Maybe it can be a vulnerability. It can be a delicious buffer overflow.
or something like that, a bunch of software bugs, normally one is not enough, you might need to chain a couple of vulnerabilities to build a weapon. One exploit that helps you
Gain a foothold in the system. And then you need to delivery this weapon. So the first three stages are melee.
getting to know your victim and how to own your victim.
You need to enter.
this machine.
Or, potentially, a physical facility. I mean, this is a very broad concept. It's elegant. I started, you know, diminishing this, but actually it has its elegance. It's pretty versatile, works across different domains.
In this course, we're gonna focus on the remaining 5 steps. We're not gonna be investing much on how you deliver this. I mean, it can be a phishing email, it can be, here's my infected USB drive, it can be many things.
But we want to study the exploitation, and especially from persistence, this other, it's called callback, data exfiltration, and lateral spread, or lateral movement.

Let's see in detail.
So, exploitation, okay, exploitation is only half selected in this highlighting.
Because I'm gonna mention at some point that there are some things that you need to do when you are quite a foothold on the machine.
But for example, you... The person who is running the vulnerable program, or is clicking on your email or something.
has not logged in as an administrator. He's using a normal account. And for Windows, on Windows, if you're used to Windows, you're gonna see that normally shows a confirmation prompt.
If you want to do an admin-level operation. Well, there are some bypasses for this, so maybe there you're gonna exploit some existing vulnerabilities. There are some... this case is called... what's the UIC for...
user authentication control, I believe. Anyway, there is these UAC bypasses just to avoid that the operating system asks you to confirm that you want to do it as administrator and stuff like that.
This is the exploitation part. Let's assume that You don't have to do much, because it's pretty standard for this level, for this course, it's a basic course on malware, so let's assume that this is taken care of, and we can focus on persistence, the first step.
So, persistence is about... Achieving and guaranteeing survivability over time.
of this threat in the system. Now, imagine that you spent months for entering a machine.
Power outage.
Reboot.
I spilled coffee on the keyboard or something.
And the computer is shut down. Now, if I achieve persistence correctly.
I'm pretty sure that whenever they're gonna boot that machine again, my malware is gonna run again.
Or there is a software bug that occasionally occurs in my malware. Well, next time it's gonna be reboot, or that this infected application is used, I know that my service, my malicious stuff, is gonna run. Okay.
So, with persistence, I try to...
ensure my survivability over time, beyond the moment and the immediate vicinity of the moment where I infect a machine.
To do that, to achieve persistence, normally you have to tamper with the operating system configuration. It may be just as easy as creating an icon in the autostart folder.
to do some sort of sophistication involving libraries, or pads, or more covert

ways, because you want to hide such changes. I mean, you can do something very nosy and visible, or you can be very careful.
And you can hide this in a way that it will go unnoticed.
Another thing that you do normally, alongside persistence, is that you want to make sure that you are the only instance of that malware that is running.
Now...
Think of, for example, you infect a machine that has already been infected, or you do persistence wrong, and you ended up
But you start 2 times.
Or 3.
in the same run, in the same instance of the machine. So, what happens is that I mean, best case, you just have a very slow malware.
that repeats stuff, and does this with poor efficiency. So, maybe it encrypts stuff twice, or...
wasted CPU cycles, but it's more likely that you're gonna break the functionality of the malware. You're gonna have some interference between the different copies of the malware. So what they do is, typically, there is a check, or a series of checks.
that discriminates whether this machine has already been infected or not. If it has been infected, a sample normally will not start another instance. Or maybe it's gonna deploy some update for the instance that is there. So there's various things that you can do, but you need to be able to recognize
If some copy of this thread is already running, because you don't want to have two concurrent executions, or let alone more.
So, yeah.
Because this, in the worst case, can harm even the malicious logic itself.
Another property... is targeting.
Now, in the past, like, when I cry, I was saying it spreads everywhere.
if I, you know, I will do this 10 years ago here, and every Windows PC was mounting some SMB drive, I will be infecting... how much is it here? 68 computers, 72.
Yeah, all infected.
Maybe there is a point, if it's a ransomware and I'm looking for money, yeah, maybe it's if a mass ransomware makes sense. There are a whole lot of other threats, which is that they want to be neat, they want to be carefully executed, they want to infect only specific people, specific victims.
This means that you need to discriminate to recognize if you are on the right victim or not. So whenever you infect a machine that is not your victim,

samples.
They have a few options. They might decide to terminate the execution, they might decide to... Do something benign.
In some cases, they might decide to behave like another malware.
And this is why, well, Think that... I mean, you infect...
10 machines that were not meant to be infected, but you can disguise yourself as another malware family.
So no one will look into this sample much. They're gonna say, well, it's some malware that I already know, I already know how to remove it, done. But when you are on the right machine, you should actually do what you were designed to do.
How do you figure the right machine? Well, there's many properties. The most basic stuff is checking region and language information, that's classic, for example, for the bank Trojans, those malwares that they try to
Steal information, financial information.
from the bank sector, and I recall analyzing in the past some malware that would execute only in Switzerland, and if the keyboard layout was French.
So the... For the French Swiss.
The banks there, the Retefe, Malbert.
was a thing at the time. In other cases, you just need to inspect the hardware and software characteristic of the machine. For example, you're targeting a government entity, and you know that they acquired a bunch of computers with that specific CPU model. If you don't have that model, you didn't...
Enter that office, you're somewhere else in the organization, you give up.
Presence of specific configuration, devices, files, you name it. Okay, the idea is that you fingerprint the machine to determine whether it's the intended machine or not.
Nevertheless, whatever you do remains visible to the eyes of a very good analyst.
So you might go undetected if there is no security measure on the machine, or if the security measures are not so good, but eventually, you're gonna get caught. Okay.
And a lot of this.
malware code.
Is the result of many years of crafting experience.
And... You know, people that develop their own methods.
and imagine that their methods are revealed.
there's many things that you could do. You could identify, for example,

signatures to recognize them fast. Whenever you get an executable that contains the string, now I'm putting it to very simple terms. Whenever I find this string, okay, that's that malware. Or I recognize some specific code that I use for encrypting stuff.
I'm gonna analyze this code and write a decryptor.
So...
When you're a malware writer, you want to protect your assets, you want to make your malware, in general, harder to analyze, you want to impede reverse engineering. One thing is.
you realize that I'm encrypting your files, and I'm malicious, and that's one thing.
But I don't want you to understand how I exactly do that, because I don't want to...
give away too much of how I work, I just want to make it harder for you. Now, if you have infinite time and skills, you're gonna reverse engineer this.
But the purpose is to deter.
People from trying that, or slow them down as much as possible.
So you can use obfuscation, which is one way of transforming the code and the data in a way that they become very difficult to analyze for humans, and even for algorithms designed for analyzing code.
And, so you can scramble, so to speak, your code, your, data structures, you can invent.
Difficult schemes and stuff, you can make it hard.
Another thing is that you might abuse other processes, and you spread your malicious code through multiple entities.
And each entity does one piece, and each of these is obfuscated, and it becomes harder to put together the pieces and analyze them as a whole.
Or, you can have evasion effects.
Which are those that I was mentioning prior. You recognize that you're on an analysis system, and you start behaving differently. So if you recognize that you're being analyzed, you show other code.
You hide your code. There are ways to do it. Now you might expect that code is just one there, that it's all there from the start. It's not. Code can be extracted while the program is running.
That's something that you don't have on a normal software. When you have a normal software, typically, at least on desktops, it's all there from the start.
Cell phone is a bit different. There are some recompilation and tricks, but here it's a whole other set of techniques.

And we're gonna see in a couple weeks, for example, code packing. Now, the common theme here is to stay undercover as much as possible.
There's a bot in this slide that says, acting on the infected machine for as long as possible.
Well, with ransomware, it might not be the case. You want to do... you want to be fast. You want to infect as much... I mean, as much documents and stuff as possible in
in little time, just because you might be interrupted, and you don't encrypt enough data. So, that specific class of malware may optimize a bit for speed, but nevertheless, we'll scramble its code, it's gonna make it hard for you.
And I believe we are on the final slide for this first half. No, another one. Right.
Okay, what's this one? Yeah.
Okay, I've stole your data.
I have owned your machine.
I have your... I have your documents, I have your credentials, what do I do with it?
First of all, I need to touch base with the attacker. I need to inform the attacker, me as the software, malicious software, I need to inform the attacker that I landed on that victim.
How do I do it? Well, normally, I use the same machinery for targeting, so I'm gonna check models, region, IP addresses, username, whatever is relevant, list of processes. I will acquire.
a description. I will sketch You know.
a description of this machine, and I'm gonna send it back to an attacker. For example, through our remote server. So I will connect to a remote server and send a packet saying, hey.
This is the information I, I attacked that computer over there, and these are the processors' IP address.
So, in case you need to...
identify it, or whatever, you have it. This stage is called beaconing, so it's whenever a malicious sample touches base with the attacker remotely to confirm that a machine has been owned, and these are the details of the machine.
Actually, you might want to do more than that. Normally, you might want to exfiltrate data, or you might want to receive instructions from the attacker.
To do what? Well, there are some malwares that are very sophisticated, like these Trojans, and there is a variant called Remote Access Trojans.
That act pretty much like, are you familiar with any desktop viewer, this

software, where you control someone's desktop? Normally you do it, I mean, for a relative of yours who needs help or something. Now, this is the same principle, but with victims.
You weaponize, you control A thousand, tens of thousands of computers.
And, I mean, it's not that you're gonna move the mouse or write something on these computers, but you might want to have to download software, coordinate to run an attack. You might do many things.
And there are some specialized malware families that receive instructions and commands from a server, and they do stuff, depending on an attacker that remotely orchestrates this behavior.
This orchestration takes place to something that is called the command and control server, also known as C2 or CNC.
Historically, it's a server that you reach at an IP address, and you do it with some communication. It can be plain HTTP, it can be SSH, it can be whatever. It can be a so-called direct link. So you open a connection, or the attacker opens one to you, and you communicate like a normal app.
In reality, we have more covert ways to do it nowadays. For example, instead of communicating directly.
We communicate through storage.
So, I have an online folder, cloud storage, where I'm gonna put something, me as the attacker.
And the malware is gonna connect there and read the command.
This is helpful, because
for the defenders, they're not gonna see who am I communicating with. They're just gonna see the command.
Okay, so making... Making the identification of the attacker harder, making even the reverse engineering harder.
Cloud is one thing. Another thing is Twitter, and similar social media, now X. Well.
You will see a random of, you know, I mean, you know that these platforms are full of trash accounts. They are fake.
But some of these fakes, they're used for data exfiltration.
And they're gonna write, clearly, one tweet at a time, some chunks of data, or they're gonna be comments.
So you may have some communication that takes place through Twitter.
There's gonna be the client, sorry, yeah, I already call it client, the malware, which acts as a client, and the remote server, the command and control server, which they're gonna communicate by posting

tweets on two different handles. So the sophistication of this thing is complex, or in other cases, you might be creative, and this with... not as creative, but I still find it creative. They use DNS queries, for example. You look up a non-existent domain.
And you're gonna... there is some way to intercept this DNS query, and in this DNS query.
you can put the beacon, for example. You can pass some information as attributes of a DNS query. You'd expect that you made, you know, some typo in looking up an address, while in reality you are leaking information. So, this is a pretty rich
You know, environment, you can be very creative with writing software.
But we're gonna have 12 weeks ahead of us to...
You know, show some of these techniques.
For... for today, this introductory part
on the malware techniques is completed. We're gonna do a good break now. I suggest we do 15 minutes, and then we finish earlier, so we don't run until 5.
We finish earlier, and we're gonna move to x86.
So I want you to be sharp because from this very high level philosophical concept we shared so far, we're gonna do a switch and we're gonna do something very technical. Okay, so enjoy your break.
Okay, part 2.
We start with the first technical topic.
Which is gonna be x86 assembly. So... As I was mentioning prior.
This is not a programming course.
It's gonna be a crash course on the structure of this assembly language.
Because we're gonna be able to... I mean, we are expected to be able to read such code and try to guess what a malware sample is doing in some specific part of its implementation. So, I assume that most of you have
heard or experimented with assembly before. Can you raise your hand if you have already taken a course on x86 assembly?
Okay... How about other architectures? ARM MIPS?
Okay, and a bunch of you never. Okay, that's good. It's a mixed class, so I will be getting different kinds of questions. For those that are already... those of you who are already familiar.
and confident in assembly. I still have some things to learn.
First, probably you're used to the Linux AT&T syntax for assembly. Well, in this course, we're gonna use, as in the malware practice, we're gonna use the Intel syntax. So, there's gonna be a change in how things are written.

So, try to approach this with a fresh mind, okay? Because some of the things that I say, and especially show.
We look a bit different than what you know.
Right, so analyzing this kind of code.
is pretty much what malware analysts do on a daily basis. Now.
You might not see this specific wording here, because this is a vintage screenshot. This is something from a very old architecture.
Let me see if this was an 8-bit or a 16... And VM...
Yeah, this should be a 16-bit architecture.
But why is this important? Well.
With... We're working with binary code.
Because it's... Pretty much the nature with which.
Most malware is developed. There is some malware that is written in scripting languages.
And it's obfuscated.
But it's still textual.
And it's run by some interpreter virtual machine, but the majority of malware is unexecutable.
is code that is compiled to execute natively without a virtual machine like the Jvm. In that sense, that kind of application virtual machine.
Or interpreter for that.
The majority of malware is binary.
You don't have to analyze the instruction for every single portion of the malware. Actually, just one part or some parts typically require this.
Close level of inspection, and there are tools that
They can help you get something in between.
low-level code.
And what you're usually used to, there will be a time where we will see those tools. We will mention them today, like decompilers, they're called.
But it's very important to be able to speak this language, because all these tools, they can go wrong. And ultimately, the only thing that counts is what the computer is gonna execute.
The computer is gonna execute.
Instructions.
And there are some ways in which malware presents and writes its instruction that is so much different than classic compiler-generated code. So, if you had taken
this assembly course, like in the engineering and computer science program,

you probably can write assembly programs that do pointer manipulation stuff, and they look like
Very similar to what GCC or Silang or MSVC do.
But with malware, you can have very different code.
Take and use some instructions, which you probably never saw before.
And we're gonna see them in one of the next lectures, that do the same thing in a compact way that is not performance efficient, a compiler will not use those instructions, but it is very effective when you're writing malware.
So, there are some handwritten sequences, for example, that don't look like compiler code.
But these are, you know, more like... Yeah, fun facts and.
exotic code. There's a whole bunch of other things. First is obfuscation.
The moment that you take a piece of software and you apply some layer of obfuscation, this will surely no longer look like the code that you've seen from compilers.
There are other things that you can do, which are very subtle.
You can have, I mean, some code, you look at it, and how do you look at it?
There is some methods that are called disassemblers.
They start from one point, which is the entry point of the program.
And this one is declared in the executable file. So, the executable file will say, the first instruction when you load this executable, you have to run the instruction located here. So, this assembler goes at this address, and tries to interpret all the instructions that are placed there.
Instructions are bytes.
So, the meaning of every byte, the interpretation of every byte, depends on what you have read so far, at least on x86.
ARM is a bit easier, but an X86 is a conditional interpretation. It's like you take a book, no?
Normally, we start reading from the first sentence, and after a period, you have a new sentence. Now, imagine that you change your encoding, and you start reading from the middle of the book.
And then I'm gonna tell you that A word is not,
You know, a consecutive sequence of letters.
and separated by a space or punctuation, I tell you that a word is made of one piece of a sequence and another piece of the next sequence. I can do this sort of stuff with malware. So I might start, for example.
I'm reading... In some order, and then I decide to jump.
From one instruction to continue somewhere else, which is not an instruction in

the current way that you're looking at the code.
Actually, this address is somewhere in the middle of another instruction, so it's like moving back to the book analogy. You're reading a sentence, and then you jump with your eyes, and you continue reading from another
word, but not from its start, from its middle, and so what's that word is gonna be? So the interpretation of what you have changes under your eyes, and that's something you never have with compiled code. Why would you do that? Well, because malware wants
Not to be analyzed.
And there is also self-modifying code, which brings, I mean, adds... kicks it up a notch on this paradigm. You have code that rewrites itself.
You start reading some code, and this is running.
And this code is gonna change the bytes, that are representing other code.
And so analyzing, it's difficult. Analyzing this is difficult. The only way, if you want to be very thorough, you have to analyze one instruction at a time, executing it. And if you don't speak assembly, good luck with that.
So there are many, many other decoys that you can use to make this code hard to analyze. And each of these techniques makes malicious code fairly different from compiler-generated code.
Now, there are tools that help, and these tools stay.
Frankly, reduce the number of places where you need to do this manual analysis.
What I do in this course, I don't teach
these tools. At least not from the start.
I've used them very occasionally. I know that the previous classes loved them at exams.
But then you will read some wonderful hallucinations in their exam reports, because they believed what the compiler was saying for that specific piece of code.
And maybe the inference that this tool was not so valid. So these tools are meant to speed up your work. If you know how to use them, what they're gonna do, they try to reconstruct a high-level representation, like a C program, for what this assembly code is doing.
And there are some sequences in assembly, they are very verbose. So if you have 10 years of experience, you just recognize them, and you say, okay, these 8 instructions are the way that it materializes a string, one character at a time, copies it on the stack. And for you, it's
No brainer. But if you're a first.

first time approaching this, it's your first time you analyze malware, you're gonna spend maybe 15 minutes on these instructions, or 5, or whatever. You spend time. So, those tools, they help.
in recognizing patterns and simplifying, but it's an imperfect process. So if you just trust the results, and you don't question them. you might end up in misleading results. So, there are limitations. First of all, because whenever you do compilation.
What do you do? You take a program that is written in C or another high-level language.
In Rust, in whatever language you have in mind.
Not the interpreted one, but, you know, let's stick with C, and this gets compiled to a binary representation. In your program, you add names for variables.
In the final program, you don't. You might have some locations we're gonna see, they're called registers or memory locations.
that they're gonna represent the variables for which you had a very nice and human-friendly name. You had variable index, variable counter, variable length.
No, you don't. The identifiers are gone.
Then you add types.
You had a struct that was called pair, and a struct that was called, I don't know, triple.
Or an array of specific objects, and you had a pointer that was characterizing that memory address with a type.
When you move to assembly, types are gone.
For the CPU, Memory is memory.
The way you read it.
we will see that characterizes memory as one type or another, but nothing prevents you from reading memory with the type of your choice. It's like when you do a cast in C. You get a void star, and you force a cast to something else. It's up to you, it's left to the programmer's responsibility. The languages, when they are compiled, they lose these properties, which are needed for the humans to write code, not for the machine to run the code. These constructs, they help us, not the machine. So, this information, whenever you compile, gets lost.
And no, you're not gonna find...
malware sample with debugging symbols in it that will help you with types or variables. Unless your instructor has been very sloppy or very generous on that day.

You're not gonna find the bug information that's gonna help you out.
Another problem is that if you apply obfuscation, the compilation is not going to remove the obfuscation. Maybe some cheap obfuscation, yes, something very simple, but something more complex, no.
And finally, there are techniques that break the obfuscation. Sorry, the compilation.
So, yeah, it's not a perfect process, and I will not show you this for at least some weeks.
So in the first weeks, we're gonna stick to learning the basics. When we are comfortable with this, for some tasks, I might consider using a decompiler, and then we're gonna validate the findings.
I mean, based on the experience in grading hundreds of exams per year, where people use primarily the compilers, I can tell you there's still some gaps, even for a basic course like this, let alone in real life with much more sophisticated malware. So, we learn the basics, and when we can, we use tools.
Right.
So... A primer, very quick primer on CPU and memory.
So, as programmers, you have these very nice abstractions. You have an unlimited number of variables.
You define types, you define... structures... For the CPU, you have...
What? The CPU is a collection of small cells, we can imagine, that hold memory of a varying size. There's a minimum size.
And then there are some other cells that contain larger sides, but typically the CPU will work already.
Well.
will support programs already with the standard size. There are some bigger sizes for optimization. For example, for multimedia operations, normally you work on... you apply some instructions in sequence on large data, so there are some specialized
circuitries that do some operations faster, but in principle, it's okay to work at smaller... with smaller chunks of data. And the CPU has a whole lot of these cells that are called registers.
So, they are... In a register, you can hold a piece of information.
And there are, in this slide, now if you take a look at the bottom right, we'll go with the pointer there.
Here...
We have some special registers that are not controlled by the programmer, but are used for debugging, for tracing how a program is behaving, so you don't

use them as a programmer, normally.
Here, on the right, this CR012, these are for the operating system.
But this other big batch here, that are called RAXRBXCD,
What do we have? BPSI 8, 9, 10, 11, all of these.
These are called general purpose registers, so these are available to the programmer.
The CPU that I'm showing you here, it's a 64-bit CPU, is more...
capable than the standard x86 CPU.
But it's backward compatible. So when I was pronouncing those names, I stopped saying R at some point. I just said A, B, C, AX, BX, CX, or 8, 9, 10.
Why? Because now we will see there are some conventions on how to refer to some registers. I can tell you, let me tell you that from 8, 9, 10, those you only have on 64-bit machines. But when we work with x86, there are
Register A, B, C, D, those are already available, okay? And with those, we should write programs.
No.
We are already moving from an abstraction where you can have as many variables as you want.
To having very few general purpose register.
And I have a slide next on that. Maybe let me just quickly show it, and then we go back to this on the language, on the contract.
So yeah, these are the registers that you will have. You have 8 registers for storing data.
8, just 8, and then you have the memory.
Now, there is one reason why you have a number of registers and you don't put everything in memory. These registers are temporary locations that are very, very fast for the CPU. So, when you need to frequently access some data.
in a function, it's better that it stays in a register, ideally. And everything that you need to access only once, you may read it from memory. It's an optimization thing.
Now.
registers, okay, how are these concepts exposed to the programmer and to a compiler? Well, there is some contract between the hardware, this nice picture here.
And what we humans can consume.
This is called Instruction Set Architecture.
That's the instruction set architecture, and then also the ABI, but this is not a compiler course, so I will keep it a bit simpler. I will talk with ISA, Instruction Set

Architecture.
It's... you can consider it an abstract model that describes what you should know as a programmer about the CPU. So it will tell you that there are 8 general-purpose registers, there are some instructions for data movement that are called in this way.
These arithmetic instructions called in another way, these logic instructions for another way, and so on.
And...
It also acts as a contract, because it says, okay, I'm telling you that there is a registered called A,
But then on the hardware, on the chip, I can implement A in whatever way I want.
So I can have 2, 3, 10 different Intel CPUs that implement this on silicon in another way.
Bots?
They will still offer me this obstruction that I have register A, register B, register C.
So it's a contract that I can implement as I want on hardware, as long as I provide the programmer the guarantees that are expected from this contract.
On the other hand, the programmer must talk to the CPU, the programmer who writes x86 assembly, or the compiler that generates Assembly from C, or from...
other languages.
Must generate... code
that is expressed in the format expected by the CPUs. So, if you want to have an addition between register A and register B, there is a very precise binary representation that you should use. There are rules, coding rules, you can imagine.
More than coding, they are... codification rules.
Are you good?
The chair is not so good. Yeah, but... Oh.
Yeah, sorry to... to the contract here. We were saying...
Yeah, there is a high-level representation, there's the add instruction, and there's the way that we write it in bits, because ultimately, the CPU speaks.
Binary.
Okay.
The... Instruction set architecture for Intel CPUs.
was designed, like, even before I was born. Well, a lot, 10 years prior than I was

born.
And then they try to capture a lot of compatibility over the years. But the very first draft.
was from the Intel 8086 CPU.
All the letter extensions that try to preserve this backward compatibility, initially they had They were, like, 16 bits.
Then they moved to 32, and now we are 64-bit.
So, x86 is 32-bit. What is 32 means? We will see it in a moment, when we will define what is the word size for a CPU. But...
There is backward compatibility, so all the machines that we use today.
are actually 64-bit CPUs, Which are able to run 32-bit software.
Some of you might ask now, but why all the machines are 64-bit, and I'm teaching you 32-bit assembly x86? Well...
For malware, most malware on Windows is still 32-bit.
There's a couple of reasons. One is that there's still a good, decent fraction, it is diminishing, but there is a fraction of machines out there that are only 32-bit. They have Windows 32-bit installed. Maybe even the CPU is 64, but the operating system is 32.
The other is that This is more technical, but Windows.
to support the execution, I mean, well, if you do this on Linux, it's a nightmare. If you take 64-bit Linux, and you try to run 32-bit programs.
It's not so comfortable.
But this is also because in the Linux world, you know, you are responsible for having your software right. Microsoft is preparing software for the masses.
So they're not gonna expect that, I don't know, one of my relatives will know the difference between 32 and 64. They bought a program two years ago, they update to Windows, and it no longer works. No.
They designed a very nice, I would say, subsystem that is called Windows on Windows 64. It allows you to run 32-bit programs on 64-bit Windows.
to support this backward compatibility, even at the operating system layer, I mean, the CPU can speak... 64-bit CPU can speak 32-bit assembly, no problem, but the operating system
Should be too.
You will see that the memory is different, everything is different.
This subsystem that supports the compatibility is also a whole lot of opportunities for creating or devising malware strategies.
So there's many gaps there that you can use to evade defenses, to do stuff in a covert way, so it's very convenient to be able to have 32-bit code, because

then you can tap into this Windows on Windows subsystem.
And do cover stuff that sometimes escape antivirus products, so it's very convenient. This is why I start with x86. Most malware is gonna be still x86, over time it's gonna be more and more 64-bit. But the percentages are not changing so fast.
And then, at some point, I will do a lecture on 64-bit.
There's some differences that you need to know.
But what you learn with 32-bit is enough. So, let's say with...
45 minutes crash course on X64, you will be good. You will be able to read X64, too. Okay, the lecture will be one hour and a half, but I'm saying in 45 minutes, if you love this, you could easily acquire all you need to...
move from 32 to 64. Right. What's the next slide? It's the word sides. It's this...
now, let's give a meaning to this 32, 64, 16, whatever.
Award is the National Science.
for the... for data.
that a CPU crunches. So, the CPU is designed to work natively, preferably with a data unit of a given size, and this is called word size.
Okay, so for an x86 CPU, the native word size is 32-bit.
Initially, when they did the 8086, 45-something years ago.
They were 16 bits.
So this means that a data unit, that CPU, had registers that would host 16 bits, but then they realized that with more memory available, for example, with 16 bits, you can index up to...
2 to the 16 minus 1, how much is that? 65,000. I'm not so good with math, with those things. But you don't index a lot of memory.
So even if you wanted to have an array index was not enough, you would have more RAM than what your bits would allow you to.
index. So they said, okay, hardware is getting more capable, let's move to 32.
With 32 bits, you can index, guess what? 4 gigabytes of memory.
Which nowadays might be little. It's costly. Okay, it's little, but memory have become way more costly than they used to be. But still, most machines have, even cell phones have more than 4 gigabytes. So even that.
in principle is small, at least for the operating system. Then you may say, okay, a single app can have no more, but bear with me in this reasoning. I mean, you can...
You're gonna need more bits.
in the moment that your hardware becomes more capable. Now, I will tell you that 64 is now plenty, and we will not need any upgrade anytime soon. Actually,

we are living with 48 out of the 64 we're using.
So, it's a lot.
But there is a bot, so you might expect now that the word size is 16 for these older CPUs, 32 for these not-so-old CPUs, and it's 64 for modern CPU.
But they want to keep backward compatibility, so if...
I don't know, until 20 years ago, I used to say that a word
is 32. No, I cannot say that a word is 64. And when they started defining word, actually, a word was 16 bits long.
So, the terminology. One thing is word size, okay? Word size is always the native size for the CPU, but when you say in assembly a word, a word is the original small 16-bit size. So, a word is 16 bits long.
When you say, I have an operand that takes a word, that operand is 16-bit.
If it's a double word.
Then it's 32.
So, in this course, we will be using a lot of D-word.
double word, because it's the native size, it's the word size of an x86. Moving to 64-bit, we have a quad word, Q word.
And so now we can define the common data types for these CPUs. We have byte, which is 8-bit. This, I didn't dare to say, you know, there is a byte, it's made of 8 bits, I assume that in a master.
You don't need that, and you will laugh at me, but we have Byte.
for 8, WARD for 16, DWARD for 32, and QWARD, not on this slide, because this is not 64-bit assembly, but QWARD. is 64-bit.
Okay, but memory, they always... I mean, I hope that when they taught you programming in C and stuff, they told you that, like, memory is like an infinite array.
Or even when they present the Turing machine, no. They tell you that memory is an infinite array.
And... Every... every cell is one byte, right? That's how the CPU sees memory. They have addresses, they start from zero, and they range to...
4 billion on 32-bit, and way more on 64-bit.
But one address... represents one byte. Memory is always byte level. So... How do I write 32-bit
of data to memory. In the register, no problem. In the register, if I write a number.
It has to be more than one byte, so one byte is maximum 255. How do I write 4,000?
4,000 in memory. 4,000 takes 2 bytes, at least, to represent it.

For the CPU, no problem. It will fit in a register. The CPU has its rules.
on how you want to write 32-bit in the hardware. It's left to the hardware, really.
But to memory.
This thing is made of 2 bytes.
It's like types, remember, in languages.
It's the type that tells you that memory, how you're accessing it. So if that memory is an integer.
It's gonna be for bytes.
If it's, short, it's gonna be 2 bytes. So how do we write the short?
Variable V that contains 4000 to memory. 4000 is gonna be made of 2 bytes.
In memory, what do I write? I write the first byte and the second byte, or the other way around?
depending on how I write it, I might have a problem.
The CPU... We'll need a rule for how you write The bytes that compose.
a multibyte Later. This rule is called Indianness.
Okay.
The... I think... I almost forgot the story after... after telling it so many years, it's related to the...
to a book.
Gulliver. Anyway.
the... They took the wording from that, Indianas.
And this defines the encoding. There's 2 ways. Big.
There is even a third, but the most, ones that you will see on real and popular hardware are two. They're called Little Indian and Big Indian.
Now, why little and big? Okay, here they're hinting at the criterion
Which byte do you write first?
The most significant or the least significant?
So one criterion will be Here, with the picture is better, but I moved the example to 4 bytes.
So, memory starts is an array, starts at zero, goes to 4 billions, we say it. This is cell.
The addresses are written in hexadecimal, okay? So it's 1000100110021003.
These are adjacent.
But the memory, I decide that I want to write at address 100004 bytes. So, and I want to use...
this.
ABADCAFA, but cafes.
It's a string, now that it's depicted like this, but in reality, it's a number.

In hex, every two letters represent one byte. This is why I used hexadecimal here. So these are 8 bits, 8 bits, 8 bits, 8 bits. How do I place them? So, AB is the most significant Byte.
and FE.
is the least significant byte. Most significant means is that it contains the bigger part of the number.
The one that has the higher power of tools, so to speak, when you represent it. those bits. So, if you follow Big Indian.
you should write the most significant first. So, at address 1000, Big Indian is like humans.
We write left to right from the.
Most significant to the least significant.
Intel CPUs, they like it the other way around. For efficiency and architectural reasons, efficiency within this architecture.
They write it the other way around. We should write the most significant as last, and the least significant as first. So we have to take our number.
And we have AB, AD, CA, and FA.
And we have to swap.
But be careful, you don't have to...
flip all the letters. You have to move them in pairs.
So... These are 8 bits.
It is the same 8 bits, they are written in the same way here. Here and here, they are the same bits, they're not rotated, flipped, or anything, it's the same, they're just translated. 8 at a time.
So, this is the first thing
that is really necessary to establish, and it's architecture-dependent, on x86, you're gonna have the little Indian, and this means that when we're gonna read in memory, and we're gonna show it in the debugger in this tool.
we're gonna see that if we access numbers, they are written in a way that is flipped compared to our human perception. But if we inspect the contents of registers in the debugger.
The debugger will show us to... will show them to us in the way that we humans work, because registers, they don't have Indianness.
It's an abstraction, a register, but NDNS exists, and it's only for memory.
Now, moving forward... registers, I mentioned that there are a few
Programmers use a lot of variables, but we have very few registers. You could tell me, hey, why didn't I put more?
Because they are very expensive to realize and to...

You can add more, but you might lose performance if you don't do it right.
So... there's some trade-off.
When they did the x86 architecture, 8 seemed reasonable compromise, and in this 8, you can store any sort of data that you want.
They're called general purpose registers. Then there is a special register that is called the status register, and it's the CPU that sets this register with some bits. Every bit has a specific meaning.
and showcases some state of the CPU.
For example, if you make a sum.
And this sum is bigger than the word sides, you have a so-called overflow.
So the resulting value, if you do 2 billions plus 3 billions, you cannot store 5 billions in 32-bit. You will need more space. So the computation is gonna be, so to speak, broken.
And because you have an overflow, so an overflow bit will be set as a result of making this addition, and there are other bits that are all set in the status register.
This is the CPU doing that for us. We can inspect it, but we cannot normally modify it.
There are ways with... that you can do, but not directly.
Program counter is another special register. This one is pretty much like
I mean, you know kids, when they start reading and they follow with the finger where they're reading? That's the program counter.
It's... Telling the CPU, you're here, and this is the next word, the next instruction.
And this is... this register is like a pointer that tells you.
you're doing this instruction. In reality, the implementation of the CPU is a little bit different. By the moment that you're executing an instruction, the program counter internally is already pointing to the next one.
it points to the one that is physically adjacent to this one. But in principle, the program counter, the one that you will see in the debugger, is the current instruction. The hardware is already at the next one, but for the programmer, it's the current instruction.
And then there's many other registers. We saw that picture, there were plenty, some I didn't even comment. Those were, for example, the...
Registers for vectorized instructions that allow you to work with multiple words at once.
But we will not need them for malware, they are not used. Anyway, the general purpose registers are those we will deal with. We will deal with the status

registers for some computations, and we will deal with the program counter to control execution.
Now, the general purpose registers, they can hold any sort of data.
Which means that you can put values.
Or you can put... Pointers. Why not? You will need pointers. Otherwise, you make only simple programs.
Right.
So... There are some registers that you can use them only in their entirety.
So this means that if you access some registers, you access all the 32 bits.
Great for pointers, great for integers.
Maybe not ideal if you have a character, a char.
Or you have a D word. Sometimes you might want to use a portion of a register.
So, for A, B, C, and D, You have an opportunity.
you can access the least significant Bytes, the list 2.
The bottom, so to speak.
You can access the... Lower 16.
Or, respectively, the lowest date, or the second lowest date.
You have the opportunity to access fractions of this. So, how, why is that? Well, again, for backward compatibility.
Initially, there is... was even an 8088 CPU with 8 bits.
You could access, for example, only... you could save only 8 bits, and you will put them here.
And here, and here, and here. So they take the register, which is still A, and they add a suffix to refer to the last byte only. This suffix is L. So if you write AL, BL, CL, and DL, you're referring to just that byte.
And you can do it also for the second last.
And it's BH, CH, and DH. This H and L stay for high and low, the high part and the low part of the lower 16 bits.
Okay.
So this means that if you want, in principle, you can place a byte here and a byte here.
You can represent your variables like this.
What you cannot do is you cannot do the same thing for this part, for the part that has been extended from 16 to 32, that you cannot touch it.
Either you touch it as a whole.
How do you touch it as a whole? Well, you need the E, extend... the E, sorry, Italian.
EAX, EBX, ECX, EDX. E stands for extended. Why extended? Because the

original architecture, the 16-4-bit one.
The 16 16 bit was Just AX, BX, CX, DX. These were the registers, and they had SI, DI,
Sp. Mbp. When they widen them to 32, they added an E.
And now, with 64-bit, the E became an R. If you put an R, you get, like, in that picture, you access the full thing.
RAX, RBX, but it's still backward compatible.
So, yeah.
So, we can access these portions, we can access the entire 16 bits at once, or we can access the 32 bits
For A, B, C, and D. For the other four... It's all or nothing.
There is one more thing.
BP...
Registered BP, which we can access only as EBP, sometimes is reserved by the compiler to support the bugability of software.
It's called frame pointer.
Not just debugability, also some other applications, but imagine that it's used to support the execution, monitoring, and debugging.
So, you no longer have... 8 now, you have 7 in your availability to store data.
Worse, in reality, there is another that is always reserved. Unless you write your own operating system, or...
you do a very funny malware that, for a while, changes the semantics and then restores this register. You have the register ESP, the ESP stack pointer register, or...
Which is, I mean, in principle, is a general purpose register, but in practice is reserved. It's a pointer to your stack memory, which we will discuss soon.
This means that...
This one is for sure taken on any operating system. It has a predetermined meaning. So, bottom line, you may have 6 or 7 registers available for your program, depending on what you do with the base pointer.
Normally, when you compile with optimizations, aggressive optimizations, the compilers say, I will no longer use the BP register for the frame pointer, I will use it for the data.
Because I want to make the program faster. I want to reduce the number of times the program needs to access memory. I will pack more stuff inside the registers.
But regardless, stack pointer is reserved. Now, just a curiosity, there are some software protection mechanisms that I've seen in the past that they use the

stack pointer as a normal variable.
They put numbers in it, and then they restore the original value. If you try that with a decompiler, good luck. It's gonna break in an instant, because a decompiler respects that the stack pointer is used to index memory.
But if you use it for other stuff, that analysis is gone.
Let's see the slide, I think I have it next on memory. I want to... yeah, one more and then memory, and then we finish the lecture. Yeah, this is nice to show.
This is what I was mentioning prior, that the CPU
Actually, has the program counter already
one step ahead. Why is that? Well, the CPU works
with a method that is called fetch.
The Q. The code execute.
On x86, Instructions that do not have a predetermined length.
Instructions have a variable length.
So, depending on how many bytes you need to encode some functionality.
One byte might be enough.
There are others that might take up to 17, if I'm not wrong. You may have up to 17 byte long instructions on x86.
This means that you don't know when you start
And looking at code, you start analyzing here. The CPU is told, jump here, go here, and run the code.
You know what is the current instruction, you don't know which is the next one until you have decoded the current instruction, because this variable length
Depends on where the current word ends, the current, sorry, yeah, instruction.
I was thinking of the book analogy.
So you need to decode.
It's like, imagine, now I want to do, again, the book analogy. You remove all the spaces from a book. Only if you read, you might figure where each word is ending, by analyzing the semantics.
And this encoding is careful, as there is no ambiguity.
So you cannot have that a sequence, this is 3, or is it 5 now? No.
The byte sequences are always discriminate the length.
But this means that the CPU must process the current instruction, all the bytes, and the code. What is the current instruction? It will see, okay. Maybe I read all 17, because it's the maximum that I can read.
And I know that the current one is 3 bytes long, so I can already update the program counter to point to what? To where I was now, plus 3.
And if I go to a 7-byte instruction, next time I go program counter plus 7.

Instructions, yeah, here I wrote 15. I was pretty sure it was 17. I should look it up. Maybe my memory is better than the slide. Anyway.
What they have... Yeah, maybe it's for the media tolerance. That might be that one.
This traction can be followed by some constants.
So, one thing is if an instruction tells
add register A with register B, so it's add EAX with EBX. It might be also add EAX with 20.
How do I write 20?
Or how do I write in the first place move to end into register A? Well, next to the instruction, I need
It's called the immediate operand. So whenever I have a constant, I should place this constant next to the instruction.
And there are some very specific rules for doing this.
So, the operation... Per se, the addition.
is given by the opcode part.
Of the instruction operation code of this binary sequence.
So, there are some opcodes for addition. Why did I use the plural here? Because there are multiple implementations of the addition, there is one implementation that is at register A with register B. So...
Whenever a source of a destination of an instruction is registered, the opcode changes, and there is a different opcode for each combination. It's not that the hardware is so complex in this, the hardware is fairly simple. There are some tables, if you... maybe once I will show you the Intel manual.
On that one, you will see...
that it's... they are, you know, they are enumerated in sequence, but it makes sense. This is a very elegant language, and you need... you will be able to discriminate. That's the opcode here, B8, it's for the move into register A.
And look, we have 01000000. What is that? Is number constant 1, but written in little endian.
Because we have to write the least significant bytes first. So if you find, moving to register A, the number 1, you're gonna find that B801.
If instead of register A, It's another register, it's DI, register EDI, it's not B8, it's BF.
Now, you don't have to write that. There is... there's clear tables that do this translation, the assembler tool will do it for you, but the hardware speaks in that way, and it's all part of the instruction set architecture.
I want to end with this slide.

I will just go over it briefly, and the next time we start over from this, this is how the memory looks like.
I need to scroll just a bit better, and beware that this is different between Linux and Windows. There are some... Slight differences.
But, it's similar. So, the lowest address is 0.
And memory grows this way.
Now, here, at the bottom, we have 7FFFFF.
So, all F, if it was FFFFF, means all one. Okay, full of one. But we have 7F.
7... Is 0111.
This representation is 10 and 31 times 1.
This means that we are stopping at.
2 billion.
This is the limit address for user space.
For the other half of the memory, so from
Which is the next value after 7FFFFF? It's 8 followed all by zeros, no? 8 and 7 times 0.
From 8000 to FFFF, it's reserved for the kernel. So this part of the operating system.
That is resident, and, you know, cannot be touched by programs, not directly.
So, these 2 gigabytes... are available.
But the architecture gives me 4GB, so there is already a design choice.
operating systems decided that it's enough to give 2GB to the program and keep 2GB for the kernel. That's on Linux. On Windows, usually it's like that, but there is an option to change this partitioning and make it 3 for the program and 1 for the...
Operating system, it's an advanced feature.
But, if you want, you can do it. It's a boot pod... boot parameter.
32-bit Windows, it's not even a thing anymore, but just so you know, it exists.
And then, the memory is divided into some regions.
That are predetermined, some, which are the program image, where you have some header information, the code of the program, some strings, like literals, those with quotes.
And then you have some libraries, Windows, they're called DLL, Dynamic Link Library, and then you have other memory. This is the final thing I'll tell you.
We have the stack.
And we have the heap. Heap is this memory that you allocate on demand with primitives like malloc. You know, you request from a library an allocator.
Stack and heap on Windows, they are...

growing in opposite direction. On Linux, it's different, the positioning and stuff. But on... Windows, it's growing in this way.
I mean, also Linux goes in that way, but the positioning is a bit... yeah. Anyway. So they grow in opposite direction.
And this will be... I will leave you with the first thing that will be annoying you for the first or second laboratory, if not three, that when we look at stack addresses.
memory that is on Stark.
The more we go to lower addresses, actually, the more the thing is growing, while normally you would expect that
If you expand something, is gonna grow.
to higher addresses, but here it's gonna be the other way around. You have a great weekend ahead. We see as well with me on Friday, you will see my colleague on Tuesday.
Oh, one more thing, please. All of you that borrowed that chair, please return it to the other room. I know it's not ideal, but we really need to...
to do that. Thank you.